Отчёт по лабораторной работе №2

Первоначальна настройка git

Ситникова Диана Алексндровна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**
3.1 **Системы контроля версий. Общие понятия** 7
3.2 **Примеры использования git** 8
3.3 **Базовая настройка git** 16
3.4 **Создание ключа ssh** 18
3.5 **Верификация коммитов с помощью *PGP*** 21
3.6 **Проверка коммитов в Git** 24

4 Выполнение лабораторной работы **26**
4.1 Установка программного обеспечения (рис. 4.1) 26
4.2 Базовая настройка git (рис. 4.2) 27
4.3 Создание ключа *ssh* (рис. 4.3) 28
4.4 Создание ключа *pgp* (рис. 4.4) 28
4.5 Добавление PGP ключа в GitHub 29
4.6 Настройка автоматических подписей коммитов git (рис. 4.6) . . . 30
4.7 Настройка gh (рис. 4.7) 30

5 Выводы **37**

Список литературы **38**

Список иллюстраций

4.1 Установка программного обеспечения 26
4.2 Команды базовой настройки git 27
4.3 Создание ключа ssh по алгоритму rsa (4096 бит) 28
4.4 Создание ключа pgp 29
4.5 Отпечаток ключа 30
4.6 Настройка автоматических подписей коммитов git 30
4.7 Настройка gh 31
4.8 Сознание репозитория курса на основе шаблона 32
4.9 Настройка каталога курса 33

Список таблиц

3.1 Варианты конвертации для разных значений параметра core.autocrlf 17

1 Цель работы

- Изучить идеологию и применение средств контроля версий.
- Освоить умения по работе с git.

2 Задание

- Создать базовую конфигурацию для работы с git.
- Создать ключ SSH.
- Создать ключ PGP.
- Настроить подписи git.
- Зарегистрироваться на Github.
- Создать локальный каталог для выполнения заданий по предмету.

3 Теоретическое введение

3.1 Системы контроля версий. Общие понятия

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных.

Системы контроля версий поддерживают возможность отслеживания и раз-

решения конфликтов, которые могут возникнуть при работе нескольких человек над одним файлом. Можно объединить (слить) изменения, сделанные разными участниками (автоматически или вручную), вручную выбрать нужную версию, отменить изменения вовсе или заблокировать файлы для изменения. В зависимости от настроек блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом, привилегированный доступ только одному пользователю, работающему с файлом.

Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Кроме того, обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить.

В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным.

Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд.

3.2 Примеры использования git

- Система контроля версий Git представляет собой набор программ командной строки. Доступ к ним можно получить из терминала посредством ввода команды `git` с различными опциями.
- Благодаря тому, что Git является распределённой системой контроля версий, резервную копию локального хранилища можно сделать простым

копированием или архивацией.

3.2.1 Основные команды git

- Перечислим наиболее часто используемые команды `git`.
- Создание основного дерева репозитория:

  ```
  git init
  ```

- Получение обновлений (изменений) текущего дерева из центрального репозитория:

  ```
  git pull
  ```

- Отправка всех произведённых изменений локального дерева в центральный репозиторий:

  ```
  git push
  ```

- Просмотр списка изменённых файлов в текущей директории:

  ```
  git status
  ```

- Просмотр текущих изменений:

  ```
  git diff
  ```

- Сохранение текущих изменений:
  - добавить все изменённые и/или созданные файлы и/или каталоги:

    ```
    git add .
    ```

  - добавить конкретные изменённые и/или созданные файлы и/или каталоги:

    ```
    git add имена_файлов
    ```

  - удалить файл и/или каталог из индекса репозитория (при этом файл и/или каталог остаётся в локальной директории):

    ```
    git rm имена_файлов
    ```

- Сохранение добавленных изменений:
  - сохранить все добавленные изменения и все изменённые файлы:

    ```
    git commit -am 'Описание коммита'
    ```

  - сохранить добавленные изменения с внесением комментария через встроенный редактор:

    ```
    git commit
    ```

  - создание новой ветки, базирующейся на текущей:

    ```
    git checkout -b имя_ветки
    ```

  - переключение на некоторую ветку:

    ```
    git checkout имя_ветки
    ```

    * (при переключении на ветку, которой ещё нет в локальном репозитории, она будет создана и связана с удалённой)

  - отправка изменений конкретной ветки в центральный репозиторий:

    ```
    git push origin имя_ветки
    ```

  - слияние ветки с текущим деревом:

    ```
    git merge --no-ff имя_ветки
    ```

- Удаление ветки:
  - удаление локальной уже слитой с основным деревом ветки:

    ```
    git branch -d имя_ветки
    ```

  - принудительное удаление локальной ветки:

    ```
    git branch -D имя_ветки
    ```

  - удаление ветки с центрального репозитория:

    ```
    git push origin :имя_ветки
    ```

3.2.2 Стандартные процедуры работы при наличии центрального репозитория

- Работа пользователя со своей веткой начинается с проверки и получения изменений из центрального репозитория (при этом в локальное дерево до начала этой процедуры не должно было вноситься изменений):

  ```
  git checkout master
  git pull
  git checkout -b имя_ветки
  ```

- Затем можно вносить изменения в локальном дереве и/или ветке.

- После завершения внесения какого-то изменения в файлы и/или каталоги проекта необходимо разместить их в центральном репозитории. Для этого необходимо проверить, какие файлы изменились к текущему моменту:

  ```
  git status
  ```

- При необходимости удаляем лишние файлы, которые не хотим отправлять в центральный репозиторий.
- Затем полезно просмотреть текст изменений на предмет соответствия правилам ведения чистых коммитов:

  ```
  git diff
  ```

- Если какие-либо файлы не должны попасть в коммит, то помечаем только те файлы, изменения которых нужно сохранить. Для этого используем команды добавления и/или удаления с нужными опциями:

  ```
  git add ...
  git rm ...
  ```

- Если нужно сохранить все изменения в текущем каталоге, то используем:

  ```
  git add .
  ```

- Затем сохраняем изменения, поясняя, что было сделано:

  ```
  git commit -am "Some commit message"
  ```

- Отправляем изменения в центральный репозиторий:

  ```
  git push origin имя_ветки
  ```

  или

  ```
  git push
  ```

3.2.3 Работа с локальным репозиторием

- Создадим локальный репозиторий.
- Сначала сделаем предварительную конфигурацию, указав имя и email владельца репозитория:

```
git config --global user.name "Имя Фамилия"
git config --global  user.email "work@mail"
```

- Настроим utf-8 в выводе сообщений `git`:

```
git config --global quotepath false
```

- Для инициализации локального репозитория, расположенного, например, в каталоге `~/tutorial`, необходимо ввести в командной строке:

```
cd
mkdir tutorial
cd tutorial
git init
```

- После это в каталоге `tutorial` появится каталог `.git`, в котором будет храниться история изменений.
- Создадим тестовый текстовый файл `hello.txt` и добавим его в локальный репозиторий:

```
echo 'hello world' > hello.txt
git add hello.txt
git commit -am 'Новый файл'
```

- Воспользуемся командой `status` для просмотра изменений в рабочем каталоге, сделанных с момента последней ревизии:

```
git status
```

- Во время работы над проектом так или иначе могут создаваться файлы, которые не требуется добавлять в последствии в репозиторий. Например, временные файлы, создаваемые редакторами, или объектные файлы, создаваемые компиляторами. Можно прописать шаблоны игнорируемых при добавлении в репозиторий типов файлов в файл `.gitignore` с помощью сервисов. Для этого сначала нужно получить список имеющихся шаблонов:

```
curl -L -s https://www.gitignore.io/api/list
```

- Затем скачать шаблон, например, для C и C++

```
curl -L -s https://www.gitignore.io/api/c >> .gitignore
curl -L -s https://www.gitignore.io/api/c++ >> .gitignore
```

3.2.4 Работа с сервером репозиториев

- Для последующей идентификации пользователя на сервере репозиториев необходимо сгенерировать пару ключей (приватный и открытый):

```
ssh-keygen -C "Имя Фамилия <work@mail>"
```

- Ключи сохраняться в каталоге `~/.ssh/`.

- Существует несколько доступных серверов репозиториев с возможностью бесплатного размещения данных. Например, https://github.com/.

- Для работы с ним необходимо сначала завести на сайте https://github.com / учётную запись. Затем необходимо загрузить сгенерённый нами ранее открытый ключ.
- Для этого зайти на сайт https://github.com/ под своей учётной записью и перейти в меню *GitHub setting*.
- После этого выбрать в боковом меню *GitHub setting>SSH-ключи* и нажать кнопку *Добавить ключ*. Скопировав из локальной консоли ключ в буфер обмена:

  ```
  cat ~/.ssh/id_rsa.pub | xclip -sel clip
  ```

- Вставляем ключ в появившееся на сайте поле.
- После этого можно создать на сайте репозиторий, выбрав в меню , дать ему название и сделать общедоступным (публичным).
- Для загрузки репозитория из локального каталога на сервер выполняем следующие команды:

  ```
  git remote add origin
    ssh://git@github.com/<username>/<reponame>.git
  git push -u origin master
  ```

- Далее на локальном компьютере можно выполнять стандартные процедуры для работы с `git` при наличии центрального репозитория.

Понятие управления программными системами и инструментами разработки рассматривается в классической литературе по операционным системам [2].

3.3 Базовая настройка git

3.3.1 Первичная настройка параметров git

- Зададим имя и email владельца репозитория:

```
git config --global user.name "Name Surname"
git config --global user.email "work@mail"
```

- Настроим utf-8 в выводе сообщений git:

```
git config --global core.quotepath false
```

- Настройте верификацию и подписание коммитов git.
- Зададим имя начальной ветки (будем называть её `master`):

```
git config --global init.defaultBranch master
```

3.3.2 Учёт переносов строк

- В разных операционных системах приняты разные символы для перевода строк:
  - Windows: `\r\n` (`CR` и `LF`);
  - Unix: `\n` (`LF`);
  - Mac: `\r` (`CR`).
- Посмотреть значения переносов строк в репозитории можно командой:

```
git ls-files --eol
```

1. Параметр `autocrlf`

- Настройка `core.autocrlf` предназначена для того, чтобы в главном репозитории все переводы строк текстовых файлах были одинаковы.
- Настройка `core.autocrlf` с параметрами `true` и `input` делает все переводы строк текстовых файлов в главном репозитории одинаковыми.
  - `core.autocrlf true`: конвертация `CRLF->LF` при коммите и обратно `LF->CRLF` при выгрузке кода из репозитория на файловую систему (обычно используется в Windows).
  - `core.autocrlf input`: конвертация `CRLF->LF` только при коммитах (используются в MacOS/Linux).
- Варианты конвертации Таблица 3.1

Таблица 3.1: Варианты конвертации для разных значений параметра core.autocrlf

core.autocrlf	false	input	true
git commit	LF -> LF	LF -> LF	LF -> CRLF
	CR -> CR	CR -> CR	CR -> CR
	CRLF -> CRLF	CRLF -> LF	CRLF -> CRLF
git checkout	LF -> LF	LF -> LF	LF -> CRLF
	CR -> CR	CR -> CR	CR -> CR
	CRLF -> CRLF	CRLF -> CRLF	CRLF -> CRLF

- Установка параметра:
  - Для Windows

    ```
    git config --global core.autocrlf true
    ```

  - Для Linux

```
git config --global core.autocrlf input
```

2. Параметр `safecrlf`

   - Настройка `core.safecrlf` предназначена для проверки, является ли окончаний строк обратимым для текущей настройки `core.autocrlf`.
     - `core.safecrlf true`: запрещается необратимое преобразование `lf<->crlf`. Полезно, когда существуют бинарные файлы, похожие на текстовые файлы.
     - `core.safecrlf warn`: печать предупреждения, но коммиты с необратимым переходом принимаются.
   - Установка параметра:

     ```
     git config --global core.safecrlf warn
     ```

3.4 Создание ключа ssh

3.4.1 Общая информация

1. Алгоритмы шифрования ssh

   1. Аутентификация

      В SSH поддерживается четыре алгоритма аутентификации по открытым ключам:

      - DSA:
        - размер ключей DSA не может превышать 1024, его следует отключить;
      - RSA:
        - следует создавать ключ большого размера: 4096 бит;
      - ECDSA:

    - ECDSA завязан на технологиях NIST, его следует отключить;
  - Ed25519:
    - используется пока не везде.

2. Симметричные шифры
   - Из 15 поддерживаемых в SSH алгоритмов симметричного шифрования, безопасными можно считать:
     - `chacha20-poly1305`;
     - `aes*-ctr`;
     - `aes*-gcm`.
   - Шифры `3des-cbc` и `arcfour` потенциально уязвимы в силу использования DES и RC4.
   - Шифр `cast128-cbc` применяет слишком короткий размер блока (64 бит).

3. Обмен ключами
   - Применяемые в SSH методы обмена ключей DH (Diffie-Hellman) и ECDH (Elliptic Curve Diffie-Hellman) можно считать безопасными.
   - Из 8 поддерживаемых в SSH протоколов обмена ключами вызывают подозрения три, основанные на рекомендациях NIST:
     - `ecdh-sha2-nistp256`;
     - `ecdh-sha2-nistp384`;
     - `ecdh-sha2-nistp521`.
   - Не стоит использовать протоколы, основанные на *SHA1*.

2. Файлы ssh-ключей
   - По умолчанию пользовательские ssh-ключи сохраняются в каталоге `~/.ssh` в домашнем каталоге пользователя.
   - Убедитесь, что у вас ещё нет ключа.

- Файлы закрытых ключей имеют названия типа id_<алгоритм> (например, id_dsa, id_rsa).
  - По умолчанию закрытые ключи имеют имена:

    id_dsa

    id_ecdsa

    id_ed25519

    id_rsa

- Открытые ключи имеют дополнительные расширения .pub.
  - По умолчанию публичные ключи имеют имена:

    id_dsa.pub

    id_ecdsa.pub

    id_ed25519.pub

    id_rsa.pub

- При создании ключа команда попросит ввести любую ключевую фразу для более надёжной защиты вашего пароля. Можно пропустить этот этап, нажав Enter.
- Сменить пароль на ключ можно с помощью команды:

  ssh-keygen -p

3.4.2 Создание ключа ssh

- Ключ ssh создаётся командой:

  ssh-keygen -t <алгоритм>

- Создайте ключи:
  - по алгоритму *rsa* с ключём размером 4096 бит:

```
ssh-keygen -t rsa -b 4096
```

  - по алгоритму *ed25519*:

```
ssh-keygen -t ed25519
```

- При создании ключа команда попросит ввести любую ключевую фразу для более надёжной защиты вашего пароля. Можно пропустить этот этап, нажав `Enter`.

- Сменить пароль на ключ можно с помощью команды:

```
ssh-keygen -p
```

3.4.3 Добавление SSH-ключа в учётную запись GitHub

- Скопируйте созданный SSH-ключ в буфер обмена командой:

```
xclip -i < ~/.ssh/id_ed25519.pub
```

- Откройте настройки своего аккаунта на GitHub и перейдем в раздел `SSH and GPC keys`.

- Нажмите кнопку `ew SSH key`.

- Добавьте в поле `Title` название этого ключа, например, `ed25519@hostname`.

- Вставьте из буфера обмена в поле `Key` ключ.

- Нажмите кнопку `Add SSH key`.

3.5 Верификация коммитов с помощью *PGP*

- Как настроить PGP-подпись коммитов с помощью gpg.

3.5.1 Общая информация

- Коммиты имеют следующие свойства:
  - author (автор) — контрибьютор, выполнивший работу (указывается для справки);
  - committer (коммитер) — пользователь, который закоммитил изменения.
- Эти свойства можно переопределить при совершении коммита.
- Авторство коммита можно подделать.
- В git есть функция подписи коммитов.
- Для подписывания коммитов используется технология PGP (см. Работа с PGP).
- Подпись коммита позволяет удостовериться в том, кто является коммитером. Авторство не проверяется.

3.5.2 Создание ключа

- Генерируем ключ

```
gpg --full-generate-key
```

- Из предложенных опций выбираем:
  - тип *RSA and RSA*;
  - размер 4096;
  - выберите срок действия; значение по умолчанию — 0 (срок действия не истекает никогда).
- GPG запросит личную информацию, которая сохранится в ключе:
  - Имя (не менее 5 символов).

- Адрес электронной почты.
  * При вводе email убедитесь, что он соответствует адресу, используемому на GitHub.
- Комментарий. Можно ввести что угодно или нажать клавишу ввода, чтобы оставить это поле пустым.

3.5.3 Экспорт ключа

- Выводим список ключей и копируем отпечаток приватного ключа:

```
gpg --list-secret-keys --keyid-format LONG
```

- Отпечаток ключа — это последовательность байтов, используемая для идентификации более длинного, по сравнению с самим отпечатком ключа.
- Формат строки:

```
sec   Алгоритм/Отпечаток_ключа Дата_создания [Флаги] [Годен_до]
      ID_ключа
```

- Экспортируем ключ в формате ASCII по его отпечатку:

```
gpg --armor --export <PGP Fingerprint>
```

3.5.4 Добавление PGP ключа в GitHub

- Копируем ключ и добавляем его в настройках профиля на GitHub (или GitLab).
- Скопируйте ваш сгенерированный PGP ключ в буфер обмена:

```
gpg --armor --export <PGP Fingerprint> | xclip -sel clip
```

- Перейдите в настройки GitHub (https://github.com/settings/keys), нажмите на кнопку *New GPG key* и вставьте полученный ключ в поле ввода.

3.5.5 Подписывание коммитов git

- Подпись коммитов при работе через терминал:

```
git commit -a -S -m 'your commit message'
```

- Флаг `-S` означает создание подписанного коммита. При этом может потребоваться ввод кодовой фразы, заданной при генерации GPG-ключа.

3.5.6 Настройка автоматических подписей коммитов git

- Используя введёный email, укажите Git применять его при подписи коммитов:

```
git config --global user.signingkey <PGP Fingerprint>
git config --global commit.gpgsign true
git config --global gpg.program $(which gpg2)
```

3.6 Проверка коммитов в Git

- GitHub и GitLab будут показывать значок *Verified* рядом с вашими новыми коммитами.

3.6.1 Режим бдительности (vigilant mode)

- На GitHub есть настройка vigilant mode.
- Все неподписанные коммиты будут явно помечены как *Unverified*.
- Включается это в настройках в разделе *SSH and GPG keys*. Установите метку на *Flag unsigned commits as unverified*.

4 Выполнение лабораторной работы

4.1 Установка программного обеспечения (рис. 4.1)

4.1.1 Установка git

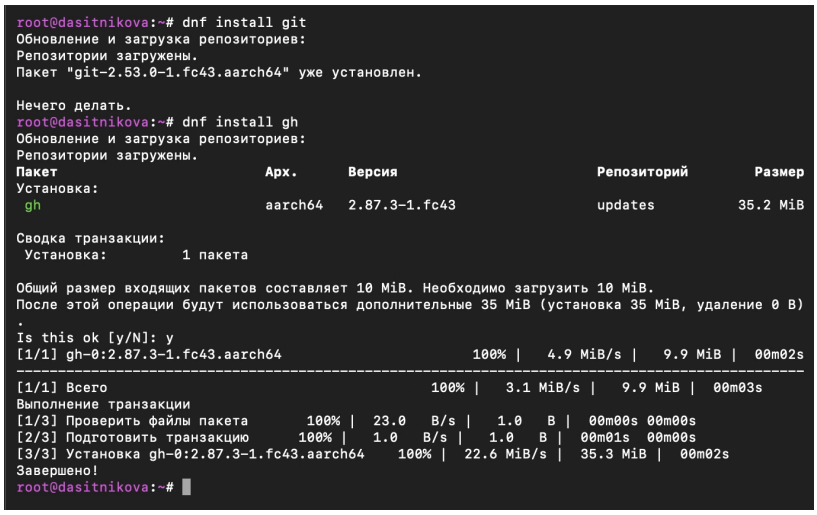
- Установим *git*:

  ```
  dnf install git
  ```

4.1.2 Установка gh

- Fedora:

  ```
  dnf install gh
  ```



```
root@dasitnikova:~# dnf install git
Обновление и загрузка репозиториев:
Репозитории загружены.
Пакет "git-2.53.0-1.fc43.aarch64" уже установлен.

Нечего делать.
root@dasitnikova:~# dnf install gh
Обновление и загрузка репозиториев:
Репозитории загружены.
Пакет                        Арх.      Версия                       Репозиторий        Размер
Установка:
 gh                          aarch64   2.87.3-1.fc43                updates          35.2 MiB

Сводка транзакции:
 Установка:        1 пакета

Общий размер входящих пакетов составляет 10 MiB. Необходимо загрузить 10 MiB.
После этой операции будут использоваться дополнительные 35 MiB (установка 35 MiB, удаление 0 B)
.
Is this ok [y/N]: y
[1/1] gh-0:2.87.3-1.fc43.aarch64                      100% |   4.9 MiB/s |   9.9 MiB |  00m02s
--------------------------------------------------------------------------------------------------
[1/1] Всего                                     100% |   3.1 MiB/s |   9.9 MiB |  00m03s
Выполнение транзакции
[1/3] Проверить файлы пакета       100% |  23.0   B/s |   1.0   B |  00m00s 00m00s
[2/3] Подготовить транзакцию      100% |   1.0   B/s |   1.0   B |  00m01s  00m00s
[3/3] Установка gh-0:2.87.3-1.fc43.aarch64    100% |  22.6 MiB/s |  35.3 MiB |  00m02s
Завершено!
root@dasitnikova:~#
```

Рисунок 4.1: Установка программного обеспечения

4.2 Базовая настройка git (рис. 4.2)

- Зададим имя и email владельца репозитория:

  ```
  git config --global user.name "Name Surname"
  git config --global user.email "work@mail"
  ```

- Настроим utf-8 в выводе сообщений git:

  ```
  git config --global core.quotepath false
  ```

- Настроим верификацию и подписание коммитов git (см. Верификация коммитов git с помощью GPG).

- Зададим имя начальной ветки (будем называть её `master`):

  ```
  git config --global init.defaultBranch master
  ```

- Параметр `autocrlf`:

  ```
  git config --global core.autocrlf input
  ```

- Параметр `safecrlf`:

  ```
  git config --global core.safecrlf warn
  ```

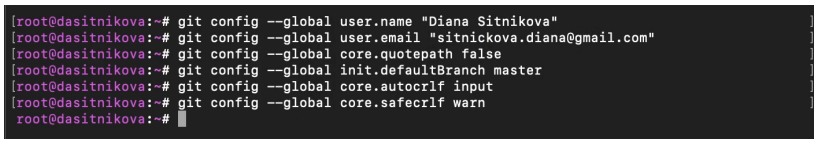


Рисунок 4.2: Команды базовой настройки git

Система контроля версий Git широко используется при разработке программного обеспечения и позволяет отслеживать изменения в проекте. Работа с инструментами командной строки подробно рассматривается в литературе по Unix-системам [1].

4.3 Создание ключа *ssh* (рис. 4.3)

- по алгоритму *rsa* с ключём размером 4096 бит:

  ```
  ssh-keygen -t rsa -b 4096
  ```

- по алгоритму *ed25519*:

  ```
  ssh-keygen -t ed25519
  ```

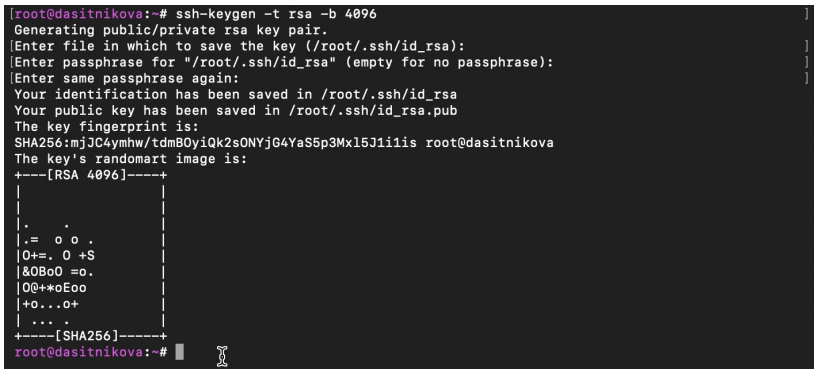


Рисунок 4.3: Создание ключа ssh по алгоритму rsa (4096 бит)

4.4 Создание ключа *pgp* (рис. 4.4)

- Генерируем ключ

  ```
  gpg --full-generate-key
  ```

- Из предложенных опций выбираем:
  - тип *RSA and RSA*;
  - размер 4096;
  - срок действия по умолчанию — 0 (срок действия не истекает никогда).

- GPG запрашивает личную информацию, которая сохранится в ключе:
  - Имя (не менее 5 символов).
  - Адрес электронной почты.
    - Вводим email соответствующий адресу на GitHub.
  - Комментарий.

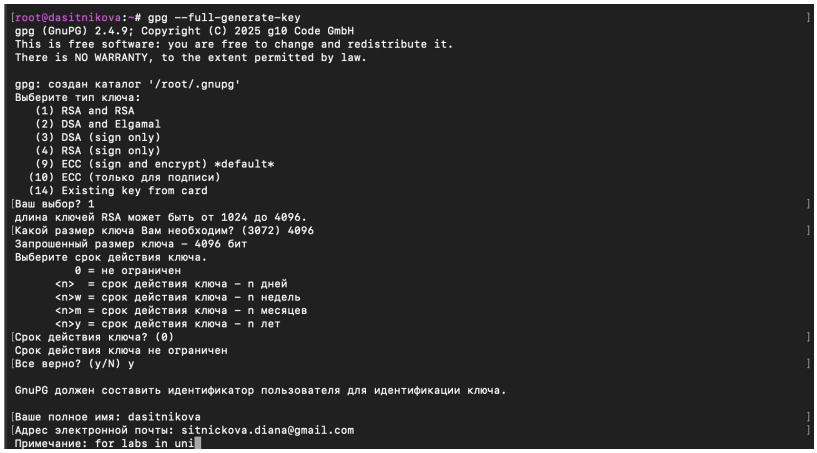


Рисунок 4.4: Создание ключа pgp

4.5 Добавление PGP ключа в GitHub

- Выводим список ключей и копируем отпечаток приватного ключа:

```
gpg --list-secret-keys --keyid-format LONG
```

- Отпечаток ключа — это последовательность байтов, используемая для идентификации более длинного, по сравнению с самим отпечатком ключа. (рис. 4.5)

- Формат строки:

```
sec   Алгоритм/Отпечаток_ключа Дата_создания [Флаги] [Годен_до]
      ID_ключа
```

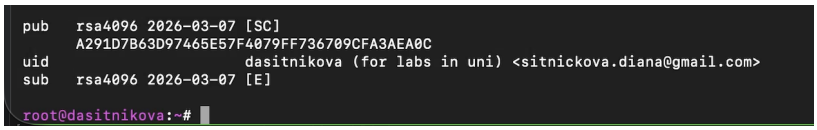


Рисунок 4.5: Отпечаток ключа

- Скопируем наш сгенерированный PGP ключ в буфер обмена:

```
gpg --armor --export <PGP Fingerprint> | xclip -sel clip
```

- Перейдем в настройки GitHub (https://github.com/settings/keys), используем кнопку *New GPG key* и вставим полученный ключ в поле ввода.

4.6 Настройка автоматических подписей коммитов git (рис. 4.6)

- Используя введёный email, укажем Git применять его при подписи коммитов:

```
git config --global user.signingkey <PGP Fingerprint>
git config --global commit.gpgsign true
git config --global gpg.program $(which gpg2)
```

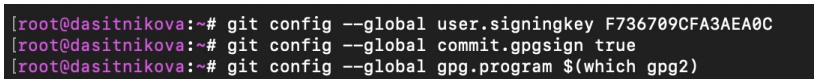


Рисунок 4.6: Настройка автоматических подписей коммитов git

4.7 Настройка gh (рис. 4.7)

- Для начала необходимо авторизоваться

```
gh auth login
```

- Утилита задаст несколько наводящих вопросов.
- Авторизоваться можно через броузер.

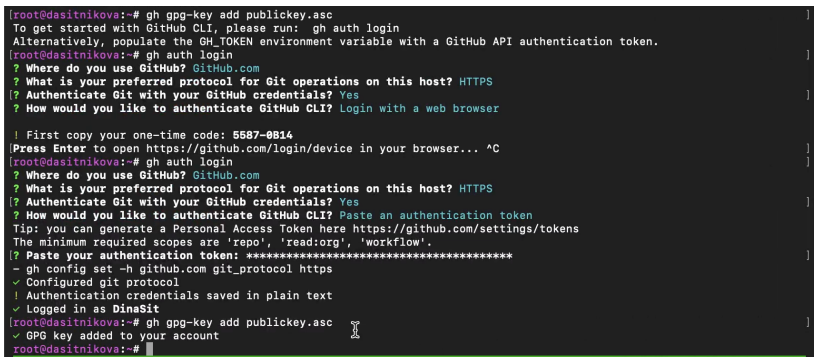


Рисунок 4.7: Настройка gh

4.7.1 Сознание репозитория курса на основе шаблона (рис. 4.8)

- Необходимо создать шаблон рабочего пространства (см. Рабочее пространство для лабораторной работы). Создание репозитория примет следующий вид:

```
mkdir -p ~/work/study/2025-2026/"Операционные системы"
cd ~/work/study/2022-2023/"Операционные системы"
gh repo create study_2022-2023_os-intro --template=yamadharma/course-directory-student-template --public
git clone --recursive git@github.com:<owner>/study_2025-2026_os-intro.git os-intro
```

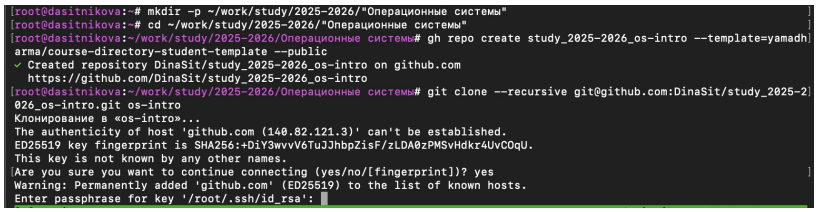


Рисунок 4.8: Сознание репозитория курса на основе шаблона

4.7.2 Настройка каталога курса (рис. 4.9)

- Перейдем в каталог курса:

  ```
  cd ~/work/study/2022-2023/"Операционные системы"/os-intro
  ```

- Удалим лишние файлы:

  ```
  rm package.json
  ```

- Создадим необходимые каталоги:

  ```
  echo os-intro > COURSE
  make prepare
  ```

- Отправим файлы на сервер:

  ```
  git add .
  git commit -am 'feat(main): make course structure'
  git push
  ```

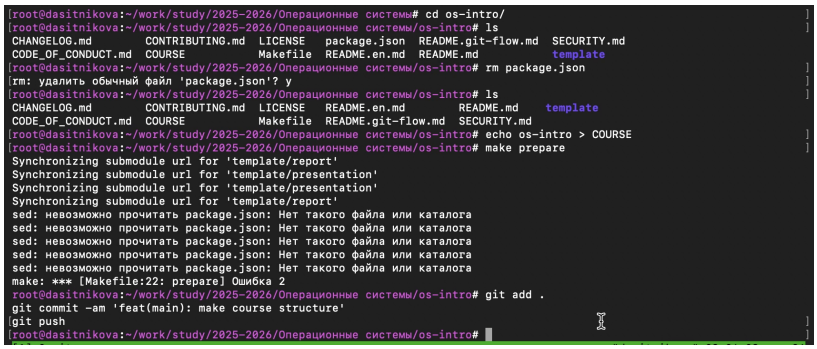


Рисунок 4.9: Настройка каталога курса

4.7.3 Контрольные вопросы

1. *Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?*

Системы контроля версий - это инструмент для отслеживания изменений в файлах и управления историей проекта. Она позволяет сохранять версии, возвращаться к предыдущим состояниям и работать нескольким разработчикам одновременно.

2. *Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.*

- Хранилище (repository) — место, где хранится проект и вся история изменений.
- Commit — зафиксированное изменение в репозитории (снимок состояния файлов).
- История — последовательность всех commit, отражающая развитие проекта.
- Рабочая копия — локальная версия файлов, с которой работает разработчик.

3. *Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.*

Централизованные — один центральный сервер с репозиторием, разработчики работают через него. Примеры: SVN, CVS. Децентрализованные (распределённые) — у каждого разработчика есть полная копия репозитория. Примеры: Git, Mercurial.

4. *Опишите действия с VCS при единоличной работе с хранилищем.*

Создавать репозиторий. Добавлять файлы в отслеживание. Вносить изменения в рабочей копии. Фиксировать изменения через commit. Просматривать историю и при необходимости откатываться.

5. *Опишите порядок работы с общим хранилищем VCS.*

Клонировать репозиторий. Вносить изменения локально. Делать commit. Получать изменения других разработчиков (pull). Отправлять свои изменения на сервер (push).

6. *Каковы основные задачи, решаемые инструментальным средством git?*

- хранение истории изменений
- совместная разработка
- управление версиями и ветвями
- откат изменений
- синхронизация с удалёнными репозиториями

7. *Назовите и дайте краткую характеристику командам git.*

`git init` — создание репозитория

`git clone` — копирование удалённого репозитория

`git add` — добавить файлы в индекс

`git commit` — сохранить изменения

`git status` — показать состояние файлов

`git log` — история commit

`git push` — отправить изменения в удалённый репозиторий

`git pull` — получить изменения

`git branch` — управление ветвями

`git merge` — объединение ветвей

8. *Приведите примеры использования при работе с локальным и удалённым репозиториями.*

Локально:

```
git init
git add main.cpp
git commit -m "initial commit"
```

Удалённый репозиторий:

```
git clone https://github.com/user/project.git
git pull
git push origin main
```

9. *Что такое и зачем могут быть нужны ветви (branches)?*

– это отдельная линия разработки. Ветви позволяют разрабатывать новые функции или исправления, не влияя на основную версию проекта, а потом объединять изменения.

10. *Как и зачем можно игнорировать некоторые файлы при commit?*

Некоторые файлы не нужно сохранять в репозитории (например, временные файлы, сборку, логи). Для этого используется файл .gitignore, в котором указываются шаблоны файлов:

```
build/
*.log
*.o
```

Git будет игнорировать такие файлы при commit.

5 Выводы

В процессе выполнения лабораторной работы мы изучили идеологию и применение средств контроля версий, а также освоили умения по работе с git.

Список литературы

1. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
2. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб. : Питер, 2015.. — 1120 с. — (Классика Computer Science).